arXiv:2503.00324v1 [cs.CR] 1 Mar 2025

DySec: A Machine Learning-based Dynamic Analysis for Detecting Malicious Packages in PyPI Ecosystem

Sk Tanzir Mehedi, Chadni Islam, Gowri Ramachandran, and Raja Jurdak

***Abstract*—Malicious Python packages make software supply chains vulnerable by exploiting trust in open-source repositories like Python Package Index (PyPI). Lack of real-time behavioral monitoring makes metadata inspection and static code analysis inadequate against advanced attack strategies such as typosquatting, covert remote access activation, and dynamic payload generation. To address these challenges, we introduce DySec, a machine learning (ML)-based dynamic analysis framework for PyPI that uses eBPF kernel and user-level probes to monitor behaviors during package installation. By capturing 36 real-time features—including system calls, network traffic, resource usage, directory access, and installation patterns—DySec detects threats like typosquatting, covert remote access activation, dynamic payload generation, and multiphase attack malware. We developed a comprehensive dataset of 14,271 Python packages, including 7,127 malicious sample traces, by executing them in a controlled isolated environment. Experimental results demonstrate that DySec achieves a 95.99% detection accuracy with a latency of <0.5s, reducing false negatives by 78.65% compared to static analysis and 82.24% compared to metadata analysis. During the evaluation, DySec flagged 11 packages that PyPI classified as benign. A manual analysis, including installation behavior inspection, confirmed six of them as malicious. These findings were reported to PyPI maintainers, resulting in the removal of four packages. DySec bridges the gap between reactive traditional methods and proactive, scalable threat mitigation in open-source ecosystems by uniquely detecting malicious install-time behaviors.**

***Index Terms*—Dynamic analysis, PyPI ecosystem, malicious detection, software supply chain/supply chain security.**

I. Introduction

OPEN-SOURCE software has become an integral part of the software supply chain. According to the Open Source Security and Risk Analysis (OSSRA) report of 2024, 96% of the 1,067 codebases scanned contain open source code, while 77% of all code in the total codebases originates from open source code [1]. However, its openness and decentralized structure add hurdles to ensuring the privacy, security, and reliability of the software supply chain [2], [3]. These software supply chain challenges are particularly evident in the ecosystem of open source packages, the modular components, that power much of today's software [1], [4]. In August 2024, more than 704,102 malicious components were identified, a 156% year-over-year increase. A significant manifestation of these challenges can be observed within the Python Package Index (PyPI) ecosystem, an official third-party software repository for Python [5], [6].

(Corresponding author: Sk Tanzir Mehedi)

Sk Tanzir Mehedi, Gowri Ramachandran, and Raja Jurdak are with the Queensland University of Technology, Brisbane, QLD 4000, Australia (e-mail: tanzir.mehedi@hdr.qut.edu.au; {g.ramachandran, r.jurdak}@qut.edu.au); Chadni Islam is with the Edith Cowan University, Joondalup, WA 6027, Australia (e-mail: c.islam@ecu.edu.au)

As of December 2024, PyPI hosted more than 590,500 packages and facilitated 1.882 billion daily downloads, serving as a cornerstone of Python's open-source ecosystem [7]–[9]. Its extensive collection of libraries and tools empowers developers, data scientists, researchers, and businesses, driving innovation across various industries, from web development and data science to machine learning and scientific computing [8]. The health and accessibility of PyPI are therefore crucial to the continued growth and success of Python and its broad user base [10]. However, its accessibility and scale have made it a prime target for malicious actors [4]. Recent reports up to July 2024 have identified 7,127 PyPI packages (1.2%) as malicious [11]–[14]. The malicious packages caused different types of attacks such as data exfiltration using AWS keys or remote API, credential theft through typosquatting, and remote code execution via dependency confusion attacks [15]–[17]. For instance, the 'Zebo-0.1.0' package periodically captures screenshots and uploads them to an attacker-controlled server using a remote API [18]. This exemplifies the broader threats within the PyPI package registry ecosystem, where users may accidentally install malicious packages. Figure 1 illustrates how attackers employ sophisticated techniques to evade security defenses. These evolving attack strategies highlight the need for robust security measures to detect and mitigate malicious packages, ensuring the integrity of the PyPI ecosystem.

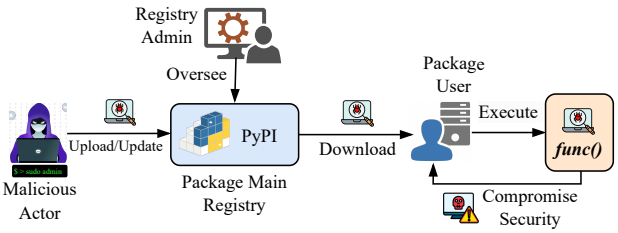


Fig. 1. Threats in the PyPI package registry ecosystem.

Existing studies aim to enhance the detection of malicious PyPI packages through various methods, including metadata analysis [19], [20], static analysis [4], [21], [22], and hybrid analysis [23], [24]. However, these methods face critical limitations in addressing evolving threats, such as detecting typosquatting, covert remote access activation, and dynamic payload generation [25]–[27]. Metadata analysis, which relies on package details, is easily bypassed by attackers using fake credentials to mimic legitimate packages. For instance, the malicious package 'python3-dateutil' deceived users by impersonating a trusted library, leading to the installation of a trojanized version that stole passwords and authentication tokens [27]. Static analysis, which inspects code without exe-

cution, struggles to detect threats due to typosquatting, remote execution, dynamically generated payloads, and multiphase attack sequences during installation—issues that often result in high false positive rates [28], [29]. For example, 'phpass', which evades detection by generating malicious payloads at install-time using encoded strings; once deployed, it exfiltrates environment variables such as AWS access keys to an attacker-controlled server [26]. Hybrid analysis, while more accurate than static or metadata methods, remains vulnerable to sandbox-aware malware that delays execution until after analysis. The package 'Pymafka' exploited this limitation by activating a keylogger immediately after installation, bypassing sandbox checks, and stealing sensitive device data [30]. Furthermore, all detection methods struggle to identify multiphase threats hidden in nested dependency chains, making them ineffective against indirect malicious dependencies [31]. Figure 2 highlights this challenge by contrasting dependency graphs of benign and malicious packages. While direct malicious dependencies (a) are detectable via existing methods, indirect dependencies (b) remain hidden within packages, exploiting install-time activation to evade detection.

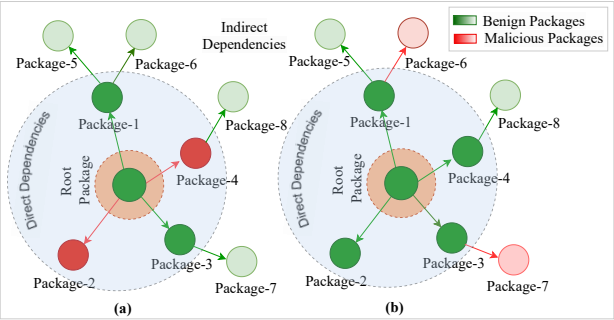


Fig. 2. Dependency graphs showing (a) direct malicious dependencies and (b) indirect malicious dependencies.

To address the limitations of existing detection methods inability to detect typosquatting, covert remote access activation, dynamic payload generation, and multiphase attack sequences, we propose an ML-based **Dy**namic **Sec**urity Analysis framework **(DySec)** for the PyPI ecosystem. DySec employs the extended Berkeley Packet Filter (eBPF) kernel and user-level probes to monitor behaviors during package installation [32]–[34]. Unlike traditional tools such as Wireshark and Sysdig, eBPF enables lightweight, real-time monitoring without requiring kernel modifications [35]. Moreover, eBPF's programmability in C or Python allows DySec to adapt dynamically to evolving security threats [32]. To effectively detect malicious PyPI packages, DySec extracts dynamic features, including system calls, network traffic, resource usage, directory access, and installation patterns. It also analyzes behavioral patterns exhibited during package installation to identify anomalies that indicate stealthy and install-time attack behaviors. To evaluate DySec's effectiveness, we created a dataset of real-world malicious PyPI package traces. These packages were executed in a controlled, isolated environment. DySec's performance was then compared against existing detection methods. The package list we considered is publicly available on DySec.io[1], ensuring reproducibility and further research. The key contributions include:

- An ML-based dynamic analysis framework, DySec detects malicious PyPI packages by monitoring install-time behavior using eBPF-based kernel and user-level probes.
- A dataset of 14,271 packages, including 7,127 malicious ones, with 36 features across six categories that capture install-time behaviors previously unexplored for malicious package detection. The dataset will be publicly available upon publication to support future research.
- A comprehensive analysis of DySec's performance demonstrates a detection accuracy of 95.99% with a latency of <0.5s, reducing false negatives by 78.65% compared to static analysis and 82.24% compared to metadata analysis, leading to the removal of four malicious packages from PyPI.

This study is structured as follows: Section II reviews existing approaches for detecting malicious packages. Section III details DySec's framework, while Section IV covers the dataset, experimental design, and implementation. Section V presents the results, and Section VI discusses the threats to validity. Finally, Section VII concludes the paper.

II. Related Work

This section provides an overview of recent approaches to malicious package detection, including metadata analysis, static analysis, hybrid techniques, and eBPF-based methods, before introducing the proposed methodology.

Metadata-based detection: Malicious package detection using metadata has gained attraction due to its efficiency and scalability [19], [20], [36]. Halder et al. [19] demonstrated effectiveness of metadata in identifying malicious PyPI packages, achieving high precision with ML models. Examples of metadata-based features include package descriptions, versioning patterns, and author profiles. Earlier work by Leckie and Yasinsac [37] established the utility of metadata for anomaly detection in security protocols, emphasizing its role in attack deduction. Mutmbak et al. [38] extended this to network security, using metadata like traffic patterns for anomaly detection. For software ecosystems, Bommarito and Bommarito [20] analyzed structural patterns in PyPI metadata, while Ohm et al. [39] identified metadata such as package naming conventions for detecting malicious npm packages. Besides research, there is a growing rise in tools designed to assess the trustworthiness of PyPI packages. For instance, PyRadar [36], Pyroma [40], and Twine [41] are proposed to evaluate and validate Python package metadata, ensuring compliance with best practices and distribution standards.

Despite its effectiveness, the ease of altering metadata undermines the reliability of metadata-based analysis. Attackers can easily manipulate metadata to introduce malicious payloads in various ways, such as falsifying maintainer details or hijacking abandoned packages [4].

Static analysis-based detection: Recent research trends have highlighted the increasing use of static code analysis for malware detection in package ecosystems such as npm

[1]https://dysec.io

and PyPI [42], [43]. For instance, Sejfia and Schäfer [43] developed an automated framework for npm packages using AST-based pattern matching and heuristics to detect malicious code snippets. Similarly, a set of studies has used static code analysis to identify vulnerabilities in PyPI packages. Ruohonen et al. [21] conducted a large-scale analysis detecting malicious patterns and vulnerabilities through code inspection, while Guo et al. [42] empirically analyzed malicious PyPI packages, uncovering common obfuscation techniques and payload delivery mechanisms. Zhang et al. [22] proposed a unified model for detecting malicious packages in npm and PyPI by analyzing behavioral sequences in code. However, Moser et al. [28] cautioned that static analysis alone has inherent limitations, such as the evasion of detection mechanisms through dynamic code generation and multiphase attack sequences. Tools like PyPI Malware Checks (PyPA) [44], OSSGadget (Microsoft) [45], and Bandit4Mal (Vu) [46] enhance traditional tools like Bandit [47] by adding specialized rules for detecting malicious code. However, their effectiveness relies on predefined patterns, leading to increased false positives, and they are unable to detect install-time payload generation or multiphase attack sequences [10], [25], [28]. Despite their utility, static tools struggle with false positive rate, typosquatting, covert remote access activation, dynamic payload generation, and multiphase attack sequences [25], [29]. As a result, researchers have increasingly turned to hybrid analysis to overcome these limitations.

Hybrid model-based detection: Recent research trends also highlight the growing use of hybrid analysis for malware detection in npm and PyPI [4], [24]. This method integrates metadata and static analysis to address limitations such as false positive rates and obfuscated code. For instance, Samaana et al. [4] integrated metadata features with static code attributes (e.g., import statements, function calls) in an ML model for PyPI, achieving higher accuracy than single-method approaches. Damodaran et al. [23] compared hybrid methods against metadata and static analysis, finding that hybrid models reduce false positives by contextualizing code behavior. Afianian et al. [24] surveyed evasion techniques against static analysis, advocating for hybrid systems to counter advanced malware. Tools like MalOSS [48], VirusTotal [13], and Packj [49] use static code inspection along with repository metadata analysis, such as activity and contributor history. They assess the risk of malicious code in software packages. None of these tools specifically address or consider install-time execution threats, which have been increasingly observed in recent years [26], [30]. To mitigate some of these limitations, hybrid analysis has been introduced, reducing false positive rates [24]. Nevertheless, challenges remain, including typosquatting, covert remote access activation, dynamic payload generation, and multiphase attack sequences [25]. As a result, researchers are increasingly turning to dynamic analysis to enhance detection capabilities and address these unresolved threats. A summary of existing tools for analyzing packages, along with their inputs and methods, is shown in Table I.

eBPF-based analysis: eBPF has emerged as a powerful tool for real-time system monitoring and is widely used across various domains to capture runtime and install-time behavior of applications [32], [33]. For instance, it has been used for malware detection, network security, and performance tracing, enabling deeper visibility into system activities without significant overhead [33]. Higuchi and Kobayashi [50] designed an eBPF-based ransomware detector using system call traces and ML, demonstrating low-latency detection. Similarly, Zhuravchak and Dudykevych [35] combined eBPF with NLP to classify ransomware behavior in real-time. Jia et al. [34] proposed programmable security policies via eBPF, enabling customizable system call monitoring. Beyond threat detection, eBPF provides a novel approach to monitoring install-time behaviors, an area where traditional methods fall short. Eitani [32] introduced bpftrace, an eBPF-powered customization tool designed to identify install-time and runtime traces of applications. Unlike traditional tools such as Wireshark and Sysdig, eBPF enables lightweight install-time monitoring without requiring kernel modifications [35]. Additionally, its programmability in C or Python allows it to dynamically adapt to evolving security threats [32].

However, traditional approaches relying on static code inspection or metadata analysis struggle to detect install-time attacks. These include typosquatting, covert remote access activation, dynamic payload generation, and multiphase attack sequences. The unique capabilities of eBPF highlight the need for frameworks that integrate eBPF with ML to analyze install-time traces for efficient malicious package detection.

TABLE I
EXISTING TOOLS FOR ANALYZING PYPI PACKAGES.

Tool Name	Input	Method
PyRadar [36]	Package	Metadata
Pyroma [40]	Package	Metadata
Twine [41]	Package	Metadata
Malware Checks [44]	Setup Script File	Static
OSSGadget [45]	Package + Artifacts	Static
Bandit4Mal [46]	Package	Static
VirusTotal [13]	Package + Artifacts	Hybrid
MalOSS [48]	Package	Hybrid
Packj [49]	Package	Hybrid
DySec [Proposed]	**Install-time Traces**	**Dynamic**

III. DYSEC FRAMEWORK

We propose DySec, a comprehensive evaluation framework for detecting malicious Python packages. It is a dynamic analysis framework that captures install-time behaviors to improve detection capabilities. Dysec comprised four key phases (i) dataset collection and validation, (ii) dynamic trace extraction, (iii) data processing and ML model evaluation, and (iv) detecting threats in real-time. The overall workflow of DySec is outlined in Figure 3.

Dataset collection and validation: There is a scarcity of publicly available malicious PyPI package datasets, as such data is often scattered across multiple sources and not centrally maintained. This lack of structured datasets makes it challenging to obtain labeled samples for research and model training. Without a standardized dataset, researchers must rely on fragmented sources, increasing the risk of incomplete or misleading data. To address this challenge, we collected malicious packages from security blogs, threat intelligence

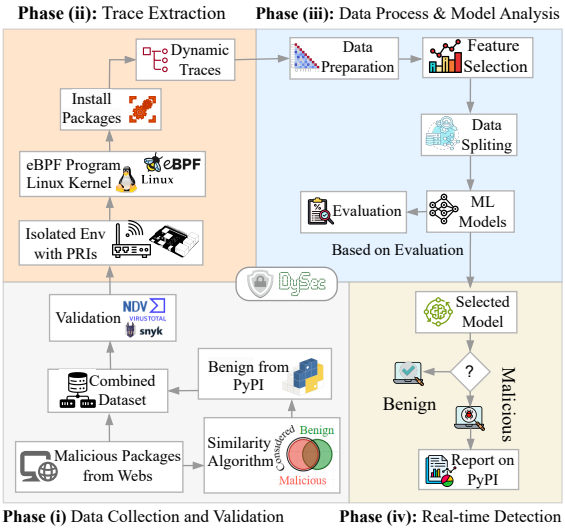


Fig. 3. The overall workflow of DySec for detecting malicious packages.

feeds, and third-party repositories. However, not all sources reliably label threats, and some packages may be benign or mislabeled. This makes maliciousness validation a critical step to ensure the integrity of our dataset. To verify the collected samples, we cross-referenced them with 62 security vendors (e.g., Kaspersky, McAfee) via the existing VirusTotal Academic API [13]. Additionally, we confirmed their absence from PyPI using its official API [7], ensuring that our dataset contains genuinely malicious packages. This step enhances dataset reliability, eliminating potentially benign samples that could otherwise skew the analysis. Also, malicious packages often mimic benign ones by using similar names, descriptions, or metadata, making detection particularly challenging.

Traditional detection methods relying solely on static analysis may struggle to distinguish between legitimate and maliciously altered versions of packages. To overcome this, we applied four similarity algorithms - String Matching (SM), Levenshtein Similarity (LS), Jaccard Similarity (JS), and Cosine Similarity (CS) [51]. These algorithms help identify benign counterparts on PyPI by measuring name and metadata similarities. To further validate this approach, the identified benign packages were verified as non-malicious using existing VirusTotal [13]. This final step establishes a reliable baseline for comparative analysis, ensuring that the dataset accurately represents both malicious and benign samples.

Dynamic trace extraction: Monitoring the behavior of potentially malicious packages poses a significant challenge, as their execution could compromise system integrity. To mitigate this risk, an isolated Raspberry Pi cluster was deployed as a secure sandbox, ensuring reproducible and unbiased analysis of malicious Python packages while preventing any impact on production systems. This controlled environment enabled comprehensive trace monitoring to capture the dynamic behavior of installed packages. Each node in the cluster integrated eBPF into the Linux Kernel to capture six categories of dynamic traces during package installations, monitoring activity for 120 seconds both during and immediately after installation. Corrupted environments from installation failures were discarded, and affected packages were reinstalled in a fresh environment. Automated scripts were employed to aggregate and preprocess these traces into structured logs, optimizing data collection for further analysis. Algorithm 1 outlines the environment setup and trace extraction process for analyzing malicious Python packages in a distributed Raspberry Pi cluster. It automates secure and isolated package installation, install-time monitoring using eBPF tracing tools, and trace aggregation, ensuring efficient data collection for dynamic analysis.

Algorithm 1: Environment Setup and Trace Extraction

Input: Package, Pi $\{Pi_1, ., Pi_n\}$ with credentials
Output: Installation traces $\mathcal{T} = \{Trace_1, ..., Trace_m\}$
Precondition: All Pi run Python 3.8-3.12 isolated Env
Step 1: Environment setup
    1.1 Initialize and setup local virtual env: `venv`
    1.2 Configure device: $\mathcal{D} \leftarrow [(host_i, user_i, key_i)]_{i=1}^{n}$
    1.3 Load all the eBPF compiler tools
Step 2: Distributed package deployment
foreach *package* $\pi_j \in$ `Package/` **do**
    if $\pi_j \notin \{$`.zip`, `.tar.gz`$\}$ **then**
        `continue`
    end
    2.1 Extract π_j metadata:
    $(name_j, version_j) \leftarrow$ `parse_filename`(π_j)
    2.2 Select target device: $d_k \leftarrow \mathcal{D}$[`rand`$(|\mathcal{D}|)$]
    2.3 Establish secure channel:
    $session_{jk} \leftarrow$ `SSHConnect`$(d_k.host, d_k.user, d_k.key)$
    2.4 Deploy package: `scp` $\pi_j \rightarrow d_k$`:/tmp/`$name_j$
end
Step 3: Installation traces
while $\exists$ *active* $session_{jk}$ **do**
    3.1 On d_k: Create `venv` $name_j$ `&& active` $name_j$
    3.2 Start tracing subsystems (120 sec):
    filetop > /traces/$name_j$_filetop.log 2>&1 &
    opensnoop > /traces/$name_j$_opens.log 2>&1 &
    tcpconnect > /traces/$name_j$_tcps.log 2>&1 &
    syscall > /traces/$name_j$_syscall.log 2>&1 &
    3.3 Execute installation:
    pip install /$name_j$ | tee /$name_j$_inst.log
    3.4 Validate installation:
    if *pip list* | *grep* $name_j$ **then**
        $status_j \leftarrow$ Success
    end
    else
        $status_j \leftarrow$ Failed
        traces/$name_j$.log $\rightarrow$ /traces/$name_j$_err.log
    end
    3.5 Terminate tracing:
    kill filetop opensnoop tcpconnect syscall
end
Step 4: Trace aggregation
foreach $session_{jk}$ **do**
    4.1 Copy traces:
    scp d_k:/traces/$name_j$ $\rightarrow$ Traces/$name_j$
    4.2 Annotate metadata:
    $name_j$,$version_j$,$status_j$,d_k $\rightarrow$ Traces/data.csv
    4.3 Clean remote env for another fresh install:
    rm -rf /tmp/$name_j$ /opt/$name_j$
end
Postcondition: $\forall \pi_j \exists\ Trace_j \in \mathcal{T}$ with: secure connections

Data processing and model evaluation: The entire data processing and model evaluation pipeline was executed in

an HPC environment to handle large-scale computations efficiently. The dataset, consisting of malicious and benign package traces, was partitioned into training, validation, and testing subsets to develop models for distinguishing malicious packages from benign ones at install time. It then underwent preprocessing to remove redundant features, optimize computational efficiency, and improve detection accuracy. The ML models were then trained on the processed data, with hyperparameters fine-tuned using the validation subset to mitigate overfitting. Finally, the best-performing model was selected based on performance metrics for real-world deployment.

Detecting threats in real-time: To assess robustness against emerging threats, a secondary analysis was conducted on packages updated or disclosed between July and November, extending beyond the initial dataset collected from November 2021 to June 2024. To ensure consistency, their dynamic traces were preprocessed in the same HPC environment using the same pipeline as the training data, maintaining uniform feature extraction and transformation. The best-performing model from the previous phase was deployed for classification, identifying benign and malicious packages. Any misclassified benign packages were flagged and reported to PyPI, ensuring transparency and proactive security enforcement.

IV. DySec Design and Setup

This section provides an overview of the experimental setup, including the dataset used for analysis, and details the hyperparameters and training settings used during the study.

A. Environment Setup

The experimental setup comprised of (i) an execution environment for package installation and trace collection, (ii) a network architecture for behavioral monitoring, and (iii) an HPC cluster for large-scale ML analysis. The execution environment utilized 16 Raspberry Pi 4 devices, each running Ubuntu 24.4 LTS with Python 3.8–3.12 in virtualized environments. A private, isolated network with a dedicated router and switch was established to prevent external interference and control traffic flow. Behavioral monitoring was implemented using eBPF integrated into the Linux kernel (v6.8.0-1012-raspi), with tools such as bcc-tools, bpftool, and bpftrace for real-time tracing. For scalable ML analysis, an HPC cluster with 16-core CPUs, NVIDIA A100 GPUs, and 128 GB RAM was used to enable parallel processing and accelerate model training.

B. Dataset Overview

This study analyzes 14,271 Python packages (7,127 malicious, 7,144 benign) to identify behavioral differences during installation. The dataset includes both successfully installed package traces (88%) and those that exhibited installation failures or anomalies (12%). This approach is critical, as even unsuccessful malicious installations—such as those causing system crashes, infinite loops, shutdowns, or unexpected authentication requests—pose operational threats and security risks. Additionally, some packages failed to generate metadata during installation, despite their dependencies being installed successfully. This raises security concerns, as installed dependencies could contain malicious components, allowing adversarial payloads to persist. In some cases, installing a required version inadvertently triggered the installation of a malicious package, further compromising system security.

To capture these variations, installation behaviors are categorized into normal, compatibility, and system-related behaviors, based on install-time characteristics observed in an isolated environment, as summarized in Table II.

TABLE II
CHARACTERISTICS OF PACKAGES DURING INSTALL-TIME TRACE COLLECTION IN AN ISOLATED ENVIRONMENT USING PYTHON 3.8-3.12.

Type	Install-time Characteristics	Malicious	Benign
Normal	Successfully installed	6,184	6,352
	Unable to generate metadata	288	232
	Missing setup files	161	0
	Failed build installable wheels	231	321
Compatibility	Mismatch distribution	73	149
	Requires a different version	121	53
	Missing package version	10	0
	Unexpected auth request	7	0
	Missing installation module	23	0
	Unicode file naming	2	0
System	System freezing	19	0
	Infinity waiting	4	0
	System shutdown	1	0
	Version looping	3	0
	System prerequisites required	0	37
	Total	**7,127**	**7,144**

To comprehensively analyze these installation behaviors, eBPF-based monitoring was employed to capture real-time system interactions and detect anomalies. eBPF provides various probes for security and monitoring, enabling the detection of malicious execution patterns, unauthorized network access, and system manipulation [32]. For instance, file system monitoring probes include *opensnoop* and *filetop*, while network activity can be monitored using *tcptop* and *tcpconnect*. Additionally, system call monitoring utilizes probes like *syscount* and *syscall*, whereas memory monitoring includes *memleak* and *biolatency*, among others [32], [33].

To enhance malware detection, these eBPF features have been widely utilized in scenarios such as network intrusion detection and ransomware analysis [33], [35], [50]. However, metadata and static analysis methods have limitations in detecting sophisticated threats due to their inability to capture install-time behaviors. To address this critical gap, user and kernel-level probes were selected to analyze system activity from multiple perspectives, ensuring a comprehensive behavioral analysis for enhanced threat detection. Each of these probes is designed to capture specific aspects of execution, allowing the identification of malicious patterns that traditional static approaches may overlook. The selected eBPF probe categories for detecting malicious packages are as follows:

- **FiletopTraces**: Monitored file read/write processes (e.g., Read_Processes, Write_Data_Transfer), directly linking to missing critical files like setup.py or pyproject.toml.
- **InstallTraces**: Tracked dependency chains (direct/indirect), exposing dependency mismatches (e.g., 'No match distribution') used in dependency confusion attacks.
- **OpensnoopTraces**: Logged directory access patterns (e.g., /root, /usr, /tmp, /sys), revealing unauthorized access to restricted paths like /root/.ssh.

- **TCPTraces**: Captured network interactions (remote IP/-port access, connection states), identifying anomalous ports (e.g., port 6667) tied to malicious payload delivery.
- **SystemCallTraces**: Recorded OS-level operations (file I/O, network, security, process management), correlating with system-level sabotage (e.g., shutdowns, freezes).
- **PatternTraces**: Aggregated sequential behaviors (e.g., Pattern_4: network socket creation, Pattern_5: process creation), detecting infinite loops or version cycling.

A detailed breakdown of these features, along with dataset examples, is provided in Appendix A1, offering further insight into how these traces contribute to malicious detection.

1) Data Preparation: The install-time traces underwent a preprocessing pipeline that included data cleaning, integration, encoding, and transformation to ensure consistency and compatibility with ML models for accurate classification.

Data cleaning and integration: To ensure dataset integrity, systematic data cleaning and integration were performed, focusing on three key steps: removing duplicate entries, filtering incomplete installation traces, and standardizing paths for feature encoding. Duplicate entries, such as typosquatted malicious package variants (e.g., reverse-shell and reverse_shell), were eliminated to prevent redundancy. Incomplete installation traces caused by Raspberry Pi cluster power issues and internet disruptions were filtered out to maintain dataset reliability. Additionally, all package installations were executed across 16 Raspberry Pi devices using a homogeneous path structure, ensuring consistency in feature encoding while avoiding bias from device-specific identifiers.

Data encoding and transformation: The extracted features were categorized into two types - categorical and numerical. Categorical features included system call sequences, which serve as fundamental indicators of program behavior, capturing interactions between package installations and the operating system. These sequences provide valuable insights into behavioral traits that aid in detection. For example, system call sequences such as *newfstatat->openat->fstat* and *socket->bind->listen* frequently appear in specific operational contexts. To capture these sequential behaviors, n-grams were employed to represent contiguous sequences of n events. This approach effectively models execution patterns, enabling differentiation between benign and malicious behaviors. The system call sequences were then vectorized into frequency-based representations, allowing ML models to recognize behavioral patterns. By preserving contextual meanings, the n-grams technique enhances the ability of ML models to learn and distinguish system behaviors [52].

Numerical features, such as *Root_DIR_Access* and *Remote_Port_ Access*, exhibited significant scale variations, spanning 10^0 to 10^4 (Figure 4). Such disparities can bias ML models, as algorithms like RF and SVM often prioritize features with larger magnitudes, potentially leading to skewed performance. To mitigate this, *Minimum–Maximum Normalization* method [53] was applied, ensuring all numerical features were scaled to a comparable range while preserving their intrinsic distributions.

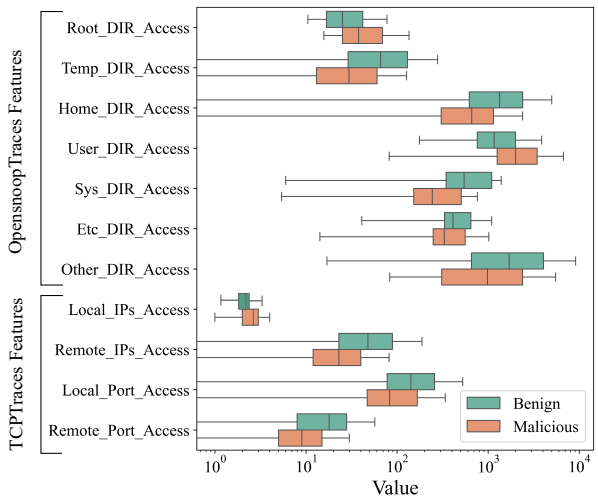


Fig. 4. A boxplot of numerical features from OpensnoopTraces and TCPTraces (before rescaling) on a logarithmic x-axis shows a significant median difference between benign and malicious packages.

C. Features Extraction and Selection

The initial set consisted of 62 Candidate Features (CFs), capturing various installation behaviors across six trace categories. However, 22 features were identified as Dependent Features (DFs) due to high correlation ($|r| > 0.50$) and were removed using *Pearson Correlation Analysis (r)* [54] to mitigate multicollinearity. This filtering step resulted in 40 Independent Features (IDFs) for further evaluation. To illustrate, in FiletopTraces, five features were retained, while four were removed due to high correlation. In contrast, PatternTraces exhibited no strong correlations ($|r| \leq 0.49$). This highlights the unique behaviors, reinforcing their importance in detecting malicious patterns.

To further refine the feature set, Importance Scores (IMS) from four ML models were used to select the most relevant features [53], [55]. Features with an IMS > 0.05 in at least one model were retained to ensure flexible feature selection. The final SEF set was determined using Equation 1.

$$SEF = \left\{ f \in CF \mid |r_f| < 0.50, \max_{m \in M} IMS_m(f) > 0.05 \right\} \cap \left\{ f \mid \max_{m \in M} IMS_m(f) > 0.08 \right\} \quad (1)$$

where:

- CF represents the initial set of candidate features.
- $|r_f| < 0.50$ removes highly correlated features.
- $IMS_m(f)$ is the importance score of feature f evaluated by model m, where M is the set of all ML models used.
- $\max_{m \in M} IMS_m(f) > 0.05$ retains a feature if at least one model assigns it an importance score > 0.05.
- $\max_{m \in M} IMS_m(f) > 0.08$ further filters the features, selecting only those with a higher importance threshold.

Table III provides an overview of the feature selection process. For example, all 10 PatternTraces features surpassed the IMS baseline threshold > 0.05 and met the Selected Engineered Features (SEF) threshold > 0.08. In contrast,

SystemCallTraces retained only 6 out of 17 features, as one feature failed to meet the SEF threshold in RF, leading to its exclusion. After filtering out correlated and low-importance features, the CombinedTraces feature set was reduced from 62 CFs to 36 SEFs, a 58% reduction. The final feature set prioritized low inter-feature correlation, high-importance scores across models, and computational efficiency.

TABLE III
FEATURE SELECTION AND REDUCTION PROCESS, FILTERING CORRELATED FEATURES AND SELECTING IMPORTANT FEATURES.

Traces Category	CF (Raw Data)	DF (|r| > 0.50)	IDF (|r| < 0.50)	IMF (IMS > 0.05)				SEF (IMS > 0.08)
				RF	DT	SVM	GB	
FiletopTraces	9	4	5	5	4	3	5	5
InstallTraces	6	3	3	3	2	2	2	3
OpensnoopTraces	12	5	7	7	6	5	7	7
TCPTraces	8	2	6	6	6	4	4	5
SystemCallTraces	17	6	11	7	6	5	6	6
PatternTraces	10	0	10	10	9	7	9	10
CombinedTraces	**62**	**22**	**40**	**37**	**35**	**28**	**33**	**36**

Note: Candidate Features (CF); Dependent Features (DF); Independent Features (IDF); Important Features (IMF); Selected Features (SEF).

D. Learning Package Maliciousness

To evaluate the effectiveness of ML models in detecting malicious packages, we assessed four models - Random Forest (RF), Decision Tree (DT), Support Vector Machine (SVM), and Gradient Boosting (GB). Each model was configured with hyperparameters that balance simplicity and performance. The RF model was configured with 100 trees and a maximum depth of 8, ensuring a trade-off between overfitting mitigation and computational efficiency. For DT, a moderate depth (max_depth=8) and conservative splitting criteria (min_samples_split=10) were applied to enhance generalizability. SVM was optimized with a linear kernel, facilitating efficient training and probabilistic outputs. GB utilized 100 weak learners with a shallow tree depth (max_depth=5) and a learning rate of 0.1 to refine sequential error correction.

The dataset was partitioned into 70% training, 15% validation, and 15% test sets. To improve generalizability, we used 5-fold stratified cross-validation to preserve class distribution across splits. The validation set was used for hyperparameter tuning (e.g., optimizing RF's n_estimators or SVM's kernel), while the test set evaluated the final performance.

V. EVALUATION

This section evaluates the overall performance of the selected models through a comprehensive analysis.

A. Evaluation Metrics

The evaluation of the selected models used four key metrics - accuracy, precision, recall, and F1-score - to assess performance across various scenarios. Accuracy measured overall correctness, while precision minimized false positives (FP), where benign packages were incorrectly classified as malicious. Recall reduced false negatives (FN), where truly malicious packages were incorrectly classified as benign. The F1-score balanced precision and recall, addressing class imbalance to provide a comprehensive assessment of model robustness and practical applicability. The evaluation was guided by the following Research Questions (RQs):

- **RQ1**: What are the specific feature sets that DySec uses for malicious package detection?
- **RQ2**: Which patterns distinguish malicious packages from benign packages?
- **RQ3**: Is DySec accurate enough to be practical?

B. RQ1: What are the specific feature sets that DySec uses for malicious package detection?

The performance evaluation of ML models across diverse feature sets presents important insights into how trace types impact malicious package detection. This section analyzes these variations, highlighting which traces enhance or degrade performance and why certain models perform better with specific trace types.

1) Feature set performance analysis: Feature set performance analysis evaluates the effectiveness of selected features in a model, measuring their impact on overall predictive power. It helps in feature selection to enhance model performance.

Combined traces analysis: The integration of six trace types - FiletopTraces, InstallTraces, OpensnoopTraces, TCPTraces, SystemCallTraces, and PatternTraces - into CombinedTraces consistently achieves the highest performance across all models, as shown in Table IV. For instance, RF attains 95.99% accuracy and 96.00% precision with CombinedTraces, surpassing all standalone traces. This improvement is due to the complementary nature of different traces, where each contributes a distinct behavioral dimension-resource monitoring, network activity, or system-level anomalies.

In detail, FiletopTraces captures resource usage trends (e.g., file read/write frequency), while OpensnoopTraces detects unauthorized directory exploration through access patterns. TCPTraces provide insights into network behavior by tracking suspicious IP or port connections, often linked to data exfiltration or command-and-control activity. However, these traces alone may not capture deeper system interactions. SystemCallTraces bridges this gap by recording OS-level activities, such as privilege escalation attempts. Meanwhile, PatternTraces aggregate sequential anomalies - such as rapid file modifications followed by network requests - revealing multi-stage attack strategies. By integrating these diverse behavioral signals, CombinedTraces provides a holistic view, significantly improving classification accuracy over individual traces. While InstallTraces contributes less to detection accuracy due to metadata overlap between benign and malicious packages, their inclusion broadens trace coverage, minimizing classification blind spots.

Standalone trace analysis: The performance of standalone traces varies significantly, affecting their effectiveness in malicious package detection. PatternTraces achieved the highest standalone performance (RF accuracy 94.62%, and F1 Score 94.61%), demonstrating their ability to capture aggregated

TABLE IV
PERFORMANCE EVALUATION RESULTS FOR ML ALGORITHMS ON DIFFERENT FEATURES; BOLD INDICATES THE OVERALL HIGHEST VALUES, UPWARD ARROW (↑) THE SECOND-BEST, DOWNWARD ARROW (↓) THE THIRD-BEST, AND UNDERLINED THE LOWEST VALUES.

ML	Metrics	Filetop Traces	Install Traces	Opensnoop Traces	TCP Traces	System Call Traces	Pattern Traces	Combined Traces
RF	Accuracy	0.9201	0.6945	0.9355	0.8374	0.8851	0.9462	**0.9599**
	Precision	0.9210	0.8028	0.9362	0.8374	0.8851	0.9495	**0.9600**
	Recall	0.9201	0.6945	0.9355	0.8374	0.8851	0.9462	**0.9599**
	F1 Score	0.9200	0.6647	0.9355	0.8374	0.8851	0.9461	**0.9602**
DT	Accuracy	0.8687	0.6950	0.9135	0.8113	0.8841	0.9462 ↓	0.9402
	Precision	0.8687	0.8084	0.9136	0.8121	0.8841	0.9495 ↓	0.9436
	Recall	0.8687	0.6950	0.9135	0.8113	0.8841	0.9462 ↓	0.9402
	F1 Score	0.8687	0.6643	0.9135	0.8111	0.8841	0.9461 ↓	0.9428
SVM	Accuracy	0.8977	0.6865	0.8005	0.8047	0.8556	0.9453	0.9528 ↑
	Precision	0.8985	0.8065	0.8165	0.8055	0.8557	0.9487	0.9530 ↑
	Recall	0.8977	0.6865	0.8005	0.8047	0.8556	0.9453	0.9528 ↑
	F1 Score	0.8976	0.6539	0.7979	0.8046	0.8556	0.9452	0.9523 ↑
GB	Accuracy	0.8738	0.6716	0.9154	0.8047	0.8561	0.9458	0.9411
	Precision	0.8742	0.7931	0.9167	0.8072	0.8562	0.9488	0.9442
	Recall	0.8738	0.6716	0.9154	0.8047	0.8561	0.9458	0.9461
	F1 Score	0.8738	0.6339	0.9153	0.8043	0.8561	0.9457	0.9435

high-level behaviors such as directory and network access sequences that distinguish malicious packages. SystemCallTraces, while effective (RF accuracy 88.51%), focuses on low-level OS interactions that may require additional context for optimal classification. FiletopTraces (RF accuracy 92.01%) and TCP Traces (83.74%) provided moderate performance by capturing resource usage and network anomalies, though their standalone effectiveness remained limited. InstallTraces is the weakest performer (RF accuracy 69.45%), likely due to significant overlap in installation dependencies and metadata between benign and malicious packages, reducing their discriminative power.

2) Algorithm-based performance analysis: Among the evaluated algorithms RF, DT, SVM, and GB, the RF-based classifier demonstrates superior accuracy with PatternTraces, achieving 94.62% by utlizing its ensemble structure to detect intricate behavioral patterns effectively. Table IV compares the performance metrics across all classifiers, highlighting RF's consistent superiority in accuracy, precision, recall, and F1-score across most traces. This advantage stems from its ability to handle high-dimensional data, mitigate overfitting through feature randomness, and aggregate diverse DTs. For example, RF's precision with CombinedTraces (96.00%) surpasses DT (94.36%), SVM (95.30%), and GB (94.42%). Its lower FP rate of 1.6% and FN rate of 2.4%, as shown in the confusion matrix (Figure 5), reinforce its reliability in minimizing classification errors and accurately detecting malicious packages.

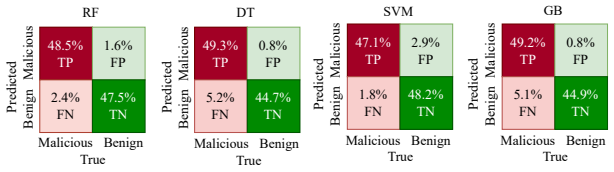


Fig. 5. Confusion matrix of selected ML models on CombinedTraces.

Unlike RF, DT shows signs of overfitting, as evidenced by its strong performance with PatternTraces at 94.62% but a slight drop to 94.02% with CombinedTraces. This suggests that DT relies more on dataset-specific patterns rather than generalizable features, leading to reduced adaptability when presented with a more diverse dataset. Finally, SVM achieves strong accuracy with CombinedTraces (95.28%) by utilizing a linear kernel, effectively distinguishing classes in high-dimensional space through optimized decision boundaries. However, its computational cost (4.4151s test time in Table V) makes it less practical for large-scale detection. GB, while effective with PatternTraces (94.58%) due to its iterative error correction, struggles with SystemCallTraces (85.61%), indicating sensitivity to trace-specific noise.

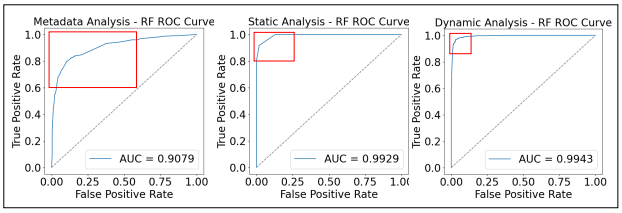


Fig. 6. ROC AUC curve comparing metadata, static, and dynamic analysis using the best-performing RF model.

3) Comparison with state-of-the-art methods: Malicious package detection requires a balance between accuracy, speed, and generalization. While ML-based metadata and static methods have been widely studied, they struggle to detect sophisticated threats due to their limited behavioral insights. This section compares DySec with the ML-based metadata method proposed by Halder et al. [19] and the ML-based static method introduced by Samaana et al. [4], highlighting key performance trade-offs and DySec's advantages in real-world scenarios. To ensure a fair comparison with state-of-the-art methods, the same model, feature selection techniques, and hyperparameters from the original research were applied to the collected metadata and static dataset.

As shown in Table V, DySec with CombinedTraces and RF outperformed metadata and static methods, achieving 95.99% accuracy, 96.02% F1 Score, and 99.37% ROC AUC (Figure 6). Its strength lies in analyzing install-time behavior, enabling the detection of subtle anomalies. This resulted in a high TP rate (48.5%) and TN rate (47.5%), while minimizing the FP rate

TABLE V
PERFORMANCE COMPARISON OF METADATA ANALYSIS (PROPOSED BY HALDER ET AL. [19]), STATIC ANALYSIS (PROPOSED BY SAMAANA ET AL. [4]), AND DYNAMIC ANALYSIS (DYSEC) USING POPULAR ML METHODS; BOLD INDICATES THE OVERALL BEST VALUES.

Methods	Data Source	Features	ML Model	Test Accu.	F1 Score	Test Time (s)	Confusion Matrix			
							TP ↑	TN ↑	FP ↓	FN ↓
Metadata Analysis [19])	File Properties	57	RF	0.8444	0.8481	0.4210	930	878	142	191
			DT	0.8393	0.8436	0.0849	928	869	144	200
			SVM	0.8047	0.8160	4.6816	927	796	145	273
			GB	0.8346	0.8425	0.0780	947	840	125	229
Static Analysis [4]	File and Source or Binary Code	21	RF	0.9514	0.9524	0.3676	1042	995	30	74
			DT	0.9514	0.9529	0.0410	1054	983	18	86
			SVM	0.9532	0.9530	3.0306	1014	1027	58	42
			GB	0.9490	0.9508	**0.0260**	1054	978	18	91
Dynamic Analysis [DySec]	Install-time Behavior	36	RF	**0.9599**	**0.9602**	0.4183	1038	1017	34	52
			DT	0.9402	0.9428	0.0588	**1055**	958	**17**	111
			SVM	0.9528	0.9523	4.4151	1009	**1031**	63	**38**
			GB	0.9411	0.9435	0.0355	1054	961	18	108

(1.6%) and FN rate (2.4%), as shown in Figure 5. These results confirm DySec's effectiveness in detecting malicious packages while reducing misclassifications.

In contrast, metadata analysis, based on the method proposed by Halder et al. [19], suffered from a high FP rate of 142 (6.6%) due to its reliance on superficial file properties, leading to frequent misclassifications. Similarly, static analysis, following the approach of Samaana et al. [4], lacked install-time behavioral context, resulting in a higher FN rate of 74 (3.5%). The absence of behavioral insights weakened its ability to differentiate between malicious and benign packages, reducing overall detection accuracy.

Beyond accuracy, an effective security solution must also balance speed and scalability. DySec's RF model achieved a test time of 0.4183s, providing high accuracy with computational efficiency. While metadata analysis was the fastest, it suffered from high FP and FN rates, making it impractical for large-scale deployments. Similarly, static analysis had a slightly faster test time (0.3676s) but failed to detect install-time anomalies, leading to higher FN rates. DySec's ability to balance high accuracy, low FP/FN rates, and reasonable test times makes it well-suited for large-scale ecosystems like PyPI, where real-time detection and scalability are critical.

While speed and scalability are essential, a robust security system must also generalize well to new and evolving threats. A key strength of DySec is its ability to detect previously unseen malware by analyzing install-time execution behavior. Unlike metadata and static analysis, which rely on predefined signatures and patterns, DySec identifies suspicious execution patterns that deviate from expected behavior. As shown in Figure 6, DySec achieved the highest ROC AUC, demonstrating strong discriminatory power in detecting malicious packages. While metadata and static methods performed well on known threats, they struggled with adversarial samples, limiting their effectiveness. By leveraging install-time behavior, DySec improves its ability to detect emerging attack variations, making it a more resilient solution against sophisticated cyber threats.

C. RQ2: Which patterns distinguish malicious packages from benign packages?

Distinguishing between benign and malicious system call patterns is challenging, as some sequences appear in both contexts. The same pattern may indicate routine activity in one case and adversarial intent in another, depending on its execution context. To address this, DySec identifies unique and independent system call patterns and evaluates their contributions to classification.

To validate its effectiveness, the ten PatternTraces categories were analyzed to assess their role in differentiating benign and malicious packages. Figure 7 presents a comparative analysis of these categories, measuring their effectiveness in classification using key performance metrics. The results indicate that individual pattern categories contribute uniquely to the detection process, reinforcing their role in identifying malicious behaviors.

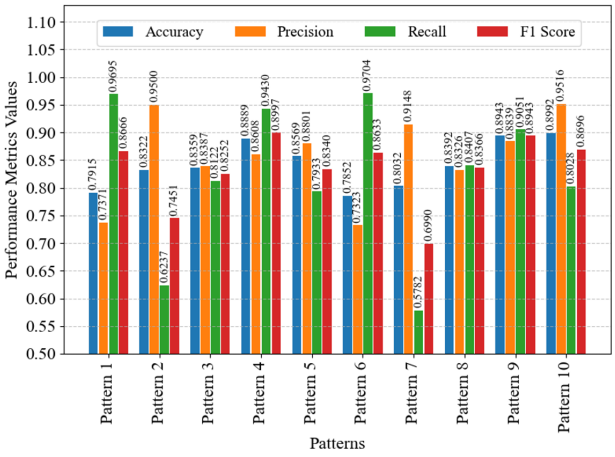


Fig. 7. Comparison metrics across SystemCall patterns using RF model.

Each of these categories consists of multiple system call patterns that represent specific system behaviors. During the evaluation of PatternTraces, 56 distinct behavioral patterns were identified, each mapped to one of ten predefined categories. These patterns play a crucial role in classification by capturing fundamental differences in system behavior between benign and malicious packages. To ensure these patterns captured unique behavioral characteristics rather than redundant information, independence was assessed using correlation analysis. The most influential system call patterns were then identified by computing feature importance scores

TABLE VI
PERFORMANCE METRICS FOR THE TOP TEN SYSTEM CALL PATTERNS.

Pattern Type and Specific Pattern	Pattern	Accuracy	Precision	Recall	F1 Score
Pattern_10: Error Handling	*newfstatat -> openat -> fstat -> error=ENOENT*	0.9954	0.9954	0.9954	0.9977
Pattern_1: File Metadata Retrieval	*newfstatat -> openat -> fstat -> lseek -> ioctl*	0.9844	0.9914	0.9922	0.9866
Pattern_2: Reading data from file	*read -> read -> read -> newfstatat*	0.9730	0.9875	0.9863	0.9863
Pattern_4: Network Socket Creation	*socket -> bind -> listen -> accept -> execve*	0.9690	0.9835	0.9843	0.9843
Pattern_6: Memory Mapping	*openat -> mmap -> ioctl -> prctl -> no-fd*	0.9679	0.9796	0.9837	0.9837
Pattern_7: File Descriptor Management	*fcntl -> fcntl -> close -> no-error -> fd=1*	0.9634	0.9757	0.9814	0.9814
Pattern_1: File Metadata Retrieval	*openat -> fstat -> ioctl*	0.9577	0.9717	0.9784	0.9784
Pattern_1: File Metadata Retrieval	*close -> newfstatat -> no-fd*	0.9286	0.9441	0.9630	0.9630
Pattern_5: Creating a New Process	*ioctl -> setresuid -> setresgid -> execve*	0.9294	0.9480	0.9634	0.9634
Pattern_9: File Locking	*openat -> fstat -> fcntl -> no-fd*	0.9259	0.9401	0.9615	0.9615

using the RF classifier's *feature_importances_* attribute, which assigns importance based on each feature's impact on predictive performance [56]. *feature_importances_* was chosen over *permutation_importance* because PatternTraces columns contain only 56 unique patterns, indicating low cardinality. While both methods are applicable, *feature_importances_* is computationally more efficient for this scenario and aligns well with the tree-based structure of the RF classifier [56].

Table VI presents the top ten system call patterns observed across benign and malicious packages. While the table does not explicitly label each pattern as benign or malicious, their system call patterns provide meaningful insights. By analyzing their occurrence, frequency, and sequence, these patterns serve as behavioral signatures, distinguishing system calls that are more prevalent in malicious activities from those typical of benign operations. A detailed analysis follows, in which each pattern is examined in depth to determine whether it is associated with benign or malicious behavior based on its behavioral signatures and classification impact. Key distinctions in behavioral signatures:

1. Error handling: Error patterns help distinguish between malicious probing and legitimate file access. Our analysis identifies frequent errors, such as error=ENOENT, as strong signals of **malicious** behavior. For instance, Pattern_10 (*newfstatat->openat->fstat->error=ENOENT*) achieves an F1 score of 0.9977. In contrast, our analysis identifies error-free such as Pattern_2: (*read->read->read->newfstatat*) as a characteristic of routine file reads in **legitimate** operations. Their high recall and precision indicate their reliability in distinguishing normal activity from malicious probing.

2. Metadata and read/write operation: Pattern_7 (*fcntl->fcntl->close->no-error->fd=1*) indicates **malicious** behavior, suggesting file descriptor manipulation for privilege escalation, with a high F1 score of 0.9814. Conversely, Pattern_1 (*newfstatat->openat->fstat->lseek->ioctl*) reflects **benign** operations, showing strong accuracy (0.9844) and precision (0.9914), typical of safe installation workflows.

3. Network and file behaviors: Pattern_4 (*socket->bind->listen->accept->execve*) highlights **malicious** behavior, indicating potential command-and-control (C2) communication, with a strong F1 score of 0.9843. Similarly, Pattern_9 (*openat->fstat->fcntl->no-fd*) signals ransomware activity, particularly through file locking techniques. In contrast, Pattern_6 (*openat->mmap->ioctl->prctl->no-fd*) represents **benign** operations, reflecting memory mapping for dependency loading. Its high precision (0.9796) and recall (0.9837) demonstrate its effectiveness in identifying normal processes and avoiding FP.

4. Structural independence: The clear separation and minimal overlap between **benign** and malicious patterns enhance classification accuracy. For example, Pattern_2 (*read->read->read->newfstatat*) remains distinct from **malicious** probing behaviors like Pattern_10 (*error=ENOENT*), as reflected in their contrasting performance metrics.

5. Robust classification: The independence of high-performing patterns enhances adaptability to novel threats. For example, Pattern_10 achieves an F1 score of 0.9977 in error-based detection, while Pattern_4 reaches 0.9843 in network-based classification. This differentiation ensures that no single pattern dominates detection, leading to a more resilient classification system.

To further refine this classification, it is essential to identify which system call patterns are strongly indicative of malicious behavior and which are benign operations. Figure 8 illustrates this distinction by presenting high-impact system call sequences that serve as behavioral signatures for threat detection in DySec. Malicious activities are strongly linked to high-impact sequences such as *newfstatat->openat->fstat->error=ENOENT* (probing for missing files) and *socket->bind->listen->accept->execve* (network-based attacks), which reflect unauthorized access, error exploitation, or command-and-control infrastructure. Other notable malicious patterns include *mmap->fork->ptrace->execve*, linked to dynamic code injection, and *openat->read->encrypt->write->rename*, suggestive of ransomware-like operations. In contrast, benign patterns such as *read->read->read->newfstatat->newfstatat* represent routine file operations, with moderate but consistent importance scores. These patterns reflect predictable workflows, such as metadata retrieval and file reads, typical of legitimate software operations. Also, benign packages often exhibit structured routines like *fstat->ioctl->lseek*, which signify error-free installation workflows and standard input/output operations.

While most system call patterns exhibit clear distinctions between benign and malicious behavior, some, such as *read->read->read->newfstatat* and *read->read->no-error*, appear in both contexts, introducing potential overlap. In benign scenarios, these patterns represent routine operations like dependency resolution and data retrieval, whereas in malicious cases, they may indicate reconnaissance or preparatory steps for data exfiltration, depending on their surrounding execution

TABLE VII
REAL-WORLD DETECTION ANALYSIS USING 500 BENIGN PACKAGES, AND 500 MALICIOUS PACKAGES; BOLD INDICATES THE BEST VALUES.

Methods	Model	Unseen Most Popular and Downloaded Packages (1,000)						Cmn. FP Package	Report on PyPI	PyPI Status
		Precision	F1 Score	TP ↑	TN ↑	FP ↓	FN ↓			
Metadata Analysis [19]	RF	0.9041	0.8536	452	393	48	107	44	No	-
	DT	0.8980	0.8376	449	377	51	123			
	SVM	0.8440	0.7932	422	358	78	142			
	GB	0.8819	0.8320	441	381	59	119			
Static Analysis [4]	RF	0.9540	0.9111	477	430	23	70	17	No	-
	DT	0.9241	0.8791	462	411	38	89			
	SVM	0.9259	0.8686	463	397	37	103			
	GB	0.9380	0.8967	469	423	31	77			
Dynamic Analysis [DySec]	RF	**0.9740**	**0.9681**	487	**481**	13	**19**	11	Report 6 Malicious Packages	Removed 4 Malicious Packages from PyPI
	DT	0.9700	0.9528	485	467	15	33			
	SVM	0.9680	0.9434	484	458	16	42			
	GB	0.9761	0.9606	**488**	472	**12**	28			

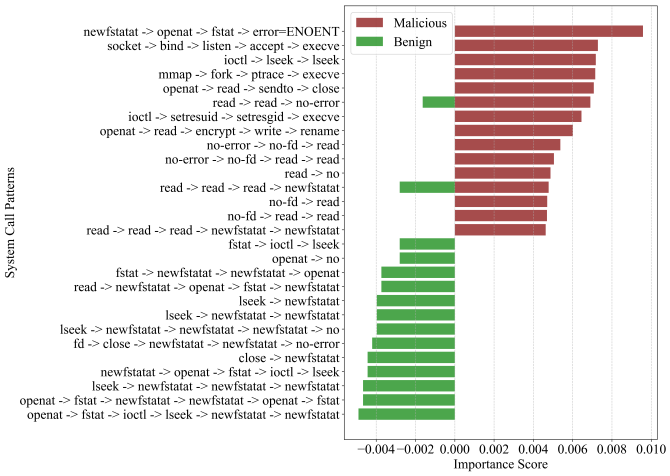


Fig. 8. Top patterns contributing to classify malicious and benign package

sequences. Despite this overlap, the structural independence of most patterns ensures that benign and malicious activities remain distinguishable. DySec effectively addresses this challenge by analyzing system call patterns within their broader execution context, enabling precise classification.

D. RQ3: Is DySec accurate enough to be practical?

To evaluate the practical applicability of the proposed DySec framework, a real-world analysis was conducted on 1,000 PyPI packages (500 benign and 500 malicious). The benign samples were selected from the latest most popular and frequently downloaded PyPI packages, ensuring they had never been exposed to the detection model [57]. The malicious samples, sourced from recently disclosed threats identified on platforms like Fortinet, BleepingComputer, The Hacker News, ReversingLabs, and Kaspersky [58]–[60], represent diverse attack patterns. To assess DySec's effectiveness against emerging threats, we performed a second evaluation on packages updated between July and November 2024.

As shown in Table VII, the RF classifier within the DySec framework achieved the best performance, with a precision of 97.40% and an F1-score of 96.81%. It outperformed both metadata and static analysis methods. It also maintained low FP (1.3%) and FN (1.9%) rates, demonstrating its reliability in accurately detecting malicious packages while minimizing disruptions to benign workflows.

Notably, the RF classifier with CombinedTraces in DySec flagged 11 commonly used benign packages as potentially malicious during testing. Upon further investigation, six exhibited malicious behaviors, including data exfiltration, port scanning, socket proxy, and unauthorized remote access. All six were reported to PyPI maintainers with supporting dynamic analysis results. Four of these - 'vermillion-0.5', 'eth-abcde-0.2.3', 'Pytonlib-0.0.0', and 'infoind-3897' - were removed from PyPI maintainers following DySec's findings. The remaining two packages - 'PySocks-1.7.1' and 'escposprinter-6.2' - were contested by PyPI maintainers. For 'PySocks-1.7.1', maintainers clarified that the package is intentionally a socket proxy server and not inherently malicious. DySec's detection of socket proxy behavior highlights its ability to flag functionalities that, while legitimate, could be repurposed for malicious activities such as anonymizing attack traffic. For 'escposprinter-6.2', maintainers noted that the package bundles NetCat (nc.exe), a network utility tool, but asserted that the binary contains no recompiled malware. DySec's alert on NetCat integration demonstrates its sensitivity to code patterns often abused in attacks, such as unauthorized network tool deployment. After validation, five additional false positive packages misclassified as malicious were confirmed benign, reaffirming DySec's low false positive rate of 1.3%. This outcome - successfully uncovering hidden threats while minimizing FN - illustrates DySec's practical effectiveness in real-world scenarios.

VI. THREATS TO VALIDITY AND LIMITATION

Internal validity: The performance of the proposed DySec framework depends on the quality and representativeness of the collected dataset. Noise in install-time traces (e.g., other IP/ port access during package installation) may impact its effectiveness by obscuring meaningful patterns. To mitigate these risks, all traces were collected in a secure, isolated Linux environment using eBPF technology. However, the reliance on a Linux-based setup introduces a limitation, as eBPF is exclusive to the Linux kernel. Additionally, configuring this isolated environment for dynamic analysis requires significant time and computational resources.

Another challenge arises when installing packages in a shared environment, as dependencies are automatically installed alongside primary packages. If these dependencies are reinstalled, the system may recognize them as already present, potentially distorting dependency resolution and analysis accuracy. To address this, each package was installed and processed in a dedicated virtual environment, minimizing dependency conflicts and ensuring consistency. Furthermore, the trace collection process generates large volumes of data, complicating feature extraction and ML model training. To address this, rigorous preprocessing steps, including data cleaning, data transformation, and feature extraction, were applied to ensure data integrity, thereby enhancing framework robustness.

External validity: This threat concerns the generalizability of findings to other ecosystems or package registries. While the framework was evaluated on a comprehensive dataset of PyPI packages (both benign and malicious), real-world scenarios may involve malware with unique characteristics not captured in the dataset. To partially address this, the framework was tested on 1,000 unseen, recently published popular packages. However, further validation across repositories such as NPM is necessary to confirm its applicability in diverse environments.

Construct validity: This threat stems from the reliance on dynamic features extracted from eBPF-based install-time traces. While these features effectively capture installation-phase behaviors, they may fail to detect delayed or runtime-specific malicious actions. Although the selected features cover a broad range of threats, sophisticated attackers employing advanced obfuscation techniques could still evade detection. Therefore, further exploration of runtime behavior monitoring is necessary to complement the install-time analysis and improve detection capabilities.

VII. Conclusion and Future Works

This study introduces DySec, an ML-based dynamic analysis framework designed to detect malicious packages within the PyPI ecosystem. It extracts features from install-time execution traces collected in a secure, isolated Linux environment using eBPF technology. To evaluate the approach, a comprehensive dataset comprising 36 dynamic, independent features was developed and tested using four ML classifiers. Among these, the RF classifier demonstrated the best performance, while the SVM classifier struggled to detect known malicious packages. DySec significantly enhances PyPI security compared to traditional metadata or static analysis methods by identifying unique malicious patterns and reducing exploitation risks. Furthermore, DySec demonstrated practical effectiveness, achieving overall low FN and FP rates when evaluated on unseen, widely used, and recently updated packages. These results underscore its potential for automated malware detection in real-world scenarios. Notably, DySec identified six packages previously classified as benign but exhibiting malicious behaviors, including data exfiltration, port scanning, and unauthorized remote access during installation. Four of these were subsequently removed by PyPI maintainers. These findings mark a critical advancement in securing open-source repositories, addressing software supply chain vulnerabilities, and fostering safer development ecosystems. In the future, we aim to bridge research and practice by developing a user-friendly platform for PyPI maintainers and developers. This platform will facilitate real-world testing, allowing users to upload package files for automated trace collection and ML-based malicious behavior prediction. Additionally, to enhance feature quality, advanced techniques such as multi-modal analysis and domain-specific behavioral modeling will be explored to detect malicious patterns more effectively.

References

[1] S. S. I. group, "2024 open source security and risk analysis (ossra) report," 2024, accessed: December 15, 2024. [Online]. Available: https://www.synopsys.com/software-integrity/resources/analyst-reports/open-source-security-risk-analysis.html

[2] H. Journal. (2025) Open source security risks. Accessed: January 1, 2025. [Online]. Available: https://www.hipaajournal.com/open-source-security-risks/

[3] W. Enck and L. Williams, "Top five challenges in software supply chain security: Observations from 30 industry and government organizations," *IEEE Security & Privacy*, vol. 20, no. 2, pp. 96–100, 2022.

[4] H. Samaana, D. E. Costa, E. Shihab, and A. Abdellatif, "A machine learning-based approach for detecting malicious pypi packages," *arXiv preprint arXiv:2412.05259*, 2024.

[5] J. S. Research, "Top jfrog security research discoveries of 2024," 2024, accessed: December 9, 2024. [Online]. Available: https://jfrog.com/blog/top-jfrog-security-research-discoveries-of-2024/?utm_source

[6] Sonatype, "State of the software supply chain 2024," 2024, accessed: December 5, 2024. [Online]. Available: https://www.sonatype.com/state-of-the-software-supply-chain/introduction?utm_source

[7] Python Software Foundation, "Pypi - the python package index," 2025, accessed: July 2, 2024. [Online]. Available: https://pypi.org/

[8] PyPI Stats, "Pypi download statistics," 2025, accessed: July 2, 2024. [Online]. Available: https://pypistats.org/packages/__all__

[9] Deps.dev, "Deps.dev - open source package insights," 2025, accessed: February 2, 2025. [Online]. Available: https://deps.dev/

[10] J. Ruohonen, K. Hjerppe, and K. Rindell, "A large-scale security-oriented static analysis of python packages in pypi," in *Proceedings of the 18th Annual International Conference on Privacy, Security and Trust (PST)*. IEEE, 2021, pp. 1–10.

[11] Snyk Security, "Snyk security - vulnerability database and security insights," 2025, accessed: May 18, 2024. [Online]. Available: https://security.snyk.io/

[12] National Institute of Standards and Technology (NIST), "National vulnerability database (nvd)," 2025, accessed: June 1, 2024. [Online]. Available: https://nvd.nist.gov/vuln

[13] VirusTotal, "Virustotal - free online virus, malware and url scanner," 2024, accessed: July 12, 2024. [Online]. Available: https://www.virustotal.com/gui/user/tanzirmehedi.qut

[14] S. S. I. Group, "Black duck software composition analysis," 2024, accessed: December 15, 2024. [Online]. Available: https://www.synopsys.com/software-integrity/software-composition-analysis-tools/languages-and-package-managers.html

[15] O. Abramovsky, "Detecting malicious packages on pypi: Malicious package on pypi use phishing techniques to hide its malicious intent," https://blog.checkpoint.com/2023/03/18/detecting-malicious-packages-on-pypi-malicious-package-on-pypi-use-phishing-techniques-to-hide-its-malicious-intent/, 2023, accessed: July 12, 2024.

[16] The Hacker News, "Pypi attack exploiting chatgpt and claude," 2024, accessed: December 8, 2024. [Online]. Available: https://thehackernews.com/2024/11/pypi-attack-chatgpt-claude.html

[17] C. S. Research, "Typosquatting attack on 'requests': One of the most popular python packages," https://zero.checkmarx.com/typosquatting-attack-on-requests-one-of-the-most-popular-python- packages-3b0a329a892d, 2024, accessed: December 6, 2024.

[18] F. T. Research, "Analyzing malicious intent in python code," 2025, accessed: January 3, 2025. [Online]. Available: https://www.fortinet.com/blog/threat-research/analyzing-malicious-intent-in-python-code

[19] S. Halder, M. Bewong, A. Mahboubi, Y. Jiang, M. R. Islam, M. Z. Islam, R. H. Ip, M. E. Ahmed, G. S. Ramachandran, and M. A. Babar, "Malicious package detection using metadata information," in *Proceedings of the ACM Web Conference 2024 (WWW '24)*, 2024, p. 11. [Online]. Available: https://doi.org/10.1145/3589334.3645543

[20] E. Bommarito and M. Bommarito, "An empirical analysis of the python package index (pypi)," *arXiv preprint arXiv:1907.11073*, 2019. [Online]. Available: https://arxiv.org/abs/1907.11073

[21] J. Ruohonen, K. Hjerppe, and K. Rindell, "A large-scale security-oriented static analysis of python packages in pypi," *arXiv preprint arXiv:2107.12699*, 2021.

[22] J. Zhang, K. Huang, B. Chen, C. Wang, Z. Tian, and X. Peng, "Malicious package detection in npm and pypi using a single model of malicious behavior sequence," *arXiv preprint arXiv:2309.02637*, 2023. [Online]. Available: https://arxiv.org/abs/2309.02637

[23] A. Damodaran, F. D. Troia, V. A. Corrado, T. H. Austin, and M. Stamp, "A comparison of static, dynamic, and hybrid analysis for malware detection," *arXiv:2203.09938*, 2022. [Online]. Available: https://arxiv.org/abs/2203.09938

[24] A. Afianian, S. Niksefat, H. R. Shahriari, and R. Jalili, "Malware dynamic analysis evasion techniques: A survey," *ACM Computing Surveys*, vol. 52, no. 6, pp. 1–28, 2020. [Online]. Available: https://dl.acm.org/doi/10.1145/3365001

[25] X. Zheng, C. Wei, S. Wang, Y. Zhao, P. Gao, Y. Zhang, K. Wang, and H. Wang, "Towards robust detection of open source software supply chain poisoning attacks in industry environments," *arXiv preprint arXiv:2409.09356*, 2024. [Online]. Available: https://arxiv.org/pdf/2409.09356

[26] C. Research, "Detecting malicious packages on pypi: Example of the 'phpass' package dynamically generating payloads," 2024, accessed: June 3, 2024. [Online]. Available: https://blog.checkpoint.com/2024/01/15/detecting-malicious-pypi-packages-phpass

[27] L. Martini, "Psa: There is a fake version of this package on pypi with malicious code," 2019, accessed: June 3, 2024. [Online]. Available: https://github.com/dateutil/dateutil/issues/984

[28] A. Moser, C. Kruegel, and E. Kirda, "Limits of static analysis for malware detection," in *Proceedings of the 23rd Annual Computer Security Applications Conference (ACSAC)*, 2007, pp. 421–430. [Online]. Available: https://seclab.nu/static/publications/acsac2007static

[29] A. M. Lungana-Niculescu, A. Colesa, and C. Oprisa, "False positive mitigation in behavioral malware detection using deep learning," in *IEEE 14th International Conference on Intelligent Computer Communication*, 2018, pp. 197–203.

[30] A. Sharma, "New 'pymafka' malicious package drops cobalt strike on macos, windows, linux," https://www.sonatype.com/blog/new-pymafka-malicious-package-drops-cobalt-strike-on-macos-windows-linux, 2022, accessed: June 3, 2024.

[31] S. S. Team, "Challenges in dependency analysis: Handling nested chains in malicious pypi packages," 2024, accessed: June 3, 2024. [Online]. Available: https://snyk.io/blog/nested-dependencies-pypi-malware

[32] A. Eitani, "Detecting ebpf malware with tracee," *Aqua Security Blog*, 2023. [Online]. Available: https://www.aquasec.com/blog/detecting-ebpf-malware-with-tracee/

[33] D. Bogle, "eBPF: A new frontier for malware," *Red Canary Blog*, 2023. [Online]. Available: https://redcanary.com/blog/threat-detection/ebpf-malware/

[34] J. Jia, Y. Zhu, D. Williams, A. Arcangeli, C. Canella, H. Franke, T. Feldman-Fitzthum, D. Skarlatos, D. Gruss, and T. Xu, "Programmable system call security with ebpf," *arXiv preprint arXiv:2302.10366*, 2023. [Online]. Available: https://arxiv.org/abs/2302.10366

[35] D. Zhuravchak and V. Dudykevych, "Real-time ransomware detection by using ebpf and natural language processing and machine learning," in *2023 IEEE 5th International Conference on Advanced ICT*, 2023.

[36] K. Gao, W. Xu, W. Yang, and M. Zhou, "PyRadar: Towards automatically retrieving and validating source code repository information for PyPI packages," *arXiv preprint arXiv:2404.16565*, 2024. [Online]. Available: https://arxiv.org/abs/2404.16565

[37] T. Leckie and A. Yasinsac, "Metadata for anomaly-based security protocol attack deduction," *IEEE Transactions on Knowledge and Data Engineering*, vol. 16, no. 9, pp. 1157–1168, 2004. [Online]. Available: https://ieeexplore.ieee.org/document/1316841/

[38] K. Mutmbak, S. Alotaibi, K. Alharbi, U. Albalawi, and O. Younes, "Anomaly detection using network metadata," *International Journal of Advanced Computer Science and Applications*, vol. 13, no. 5, 2022. [Online]. Available: https://thesai.org/Publications/ViewPaper?Volume=13&Issue=5&Code=IJACSA&SerialNo=82

[39] M. Ohm, F. Boes, C. Bungartz, and M. Meier, "On the feasibility of supervised machine learning for the detection of malicious software packages," in *Proceedings of the 17th International Conference on Availability, Reliability and Security (ARES)*. ACM, 2022, pp. 1–10. [Online]. Available: https://dl.acm.org/doi/10.1145/3538969.3544415

[40] L. Regebro, "Pyroma: Python package metadata checker," GitHub repository, 2025, accessed: May 12, 2024. [Online]. Available: https://github.com/regebro/pyroma

[41] P. P. A. (PyPA), "Twine: Utilities for interacting with pypi," GitHub repository, 2025, accessed: May 12, 2024. [Online]. Available: https://github.com/pypa/twine

[42] W. Guo, Z. Xu, C. Liu, C. Huang, Y. Fang, and Y. Liu, "An empirical study of malicious code in pypi ecosystem," *arXiv preprint arXiv:2309.11021*, 2023.

[43] A. Sejfia and M. Schäfer, "Practical automated detection of malicious npm packages," in *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. Pittsburgh, PA, USA: ACM, 2022, pp. 1681–1691. [Online]. Available: https://doi.org/10.1145/3510003.3510104

[44] P. P. A. (PyPA), "Pypi malware checks," a set of checks run against PyPI packages to detect malicious code patterns. [Online]. Available: https://github.com/pypa/warehouse/tree/main/warehouse/malware/checks

[45] Microsoft, "Ossgadget," tools to analyze OS packages across multiple ecosystems. [Online]. Available: https://github.com/microsoft/OSSGadget

[46] D.-L. Vu, "bandit4mal: A fork of bandit with enhanced detection for malicious code," an enhanced version of Bandit with additional patterns to detect malicious code. [Online]. Available: https://github.com/lyvd/bandit4mal

[47] P. C. Q. A. (PyCQA), "Bandit: A security linter for python code," a tool designed to find common security issues in Python code by analyzing the abstract syntax tree (AST). [Online]. Available: https://github.com/PyCQA/bandit

[48] D.-L. Vu *et al.*, "Maloss: An analysis platform for malicious open source software," a platform for analyzing and detecting malicious open-source packages. [Online]. Available: https://github.com/chainguard-dev/maloss

[49] P. Team, "Packj: Automated risk detection for open-source software packages," a tool that performs static and dynamic analysis to detect risks in open-source packages. [Online]. Available: https://github.com/ossillate-inc/packj

[50] K. Higuchi and R. Kobayashi, "Real-time defense system using ebpf for machine learning-based ransomware detection method," in *2023 Eleventh International Symposium on Computing and Networking Workshops*, 2023, pp. 213–219.

[51] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, p. 707, 1966.

[52] M. Z. Masud, S. Sahib, M. F. Abdollah, S. R. Selamat, and R. Yusof, "An evaluation of n-gram system call sequence in mobile malware detection," *ARPN Journal of Engineering and Applied Sciences*, vol. 11, no. 5, pp. 3122–3126, 2016. [Online]. Available: https://www.arpnjournals.org/jeas/research_papers/rp_2016/jeas_0316_3766.pdf

[53] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY: Springer, 2009.

[54] K. Pearson, "Note on regression and inheritance in the case of two parents," *Proceedings of the Royal Society of London*, vol. 58, pp. 240–242, 1895.

[55] S. T. Mehedi, A. Anwar, Z. Rahman, K. Ahmed, and R. Islam, "Dependable Intrusion Detection System for IoT: A Deep Transfer Learning Based Approach," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 1, 2023.

[56] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Feature importances with forests of trees," https://scikit-learn.org/stable/auto_examples/ensemble/plot_forest_importances.html, 2024, accessed on October 27, 2024.

[57] PyPI Stats, "Top projects on pypi," n.d., accessed on December 27, 2024. [Online]. Available: https://pypistats.org/top

[58] Fortinet, "Malicious packages hidden in pypi," 2024, accessed on December 30, 2024. [Online]. Available: https://www.fortinet.com/blog/threat-research/malicious-packages-hidden-in-pypi

[59] B. Toulas, "Malicious pypi package with 37,000 downloads steals aws keys," 2024, accessed: February 3, 2025. [Online]. Available: https://www.bleepingcomputer.com/news/security/malicious-pypi-package-with-37-000-downloads-steals-aws-keys/

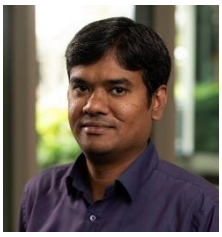
[60] Kaspersky, "Jarkastealer in pypi repository," 2024, accessed on December 30, 2024. [Online]. Available: https://www.kaspersky.com/blog/jarkastealer-in-pypi-packages/52640/



Sk Tanzir Mehedi (Graduate Student Member, IEEE) is studying toward a Ph.D. in Computer Science at Queensland University of Technology (QUT), Brisbane, Australia. He has teaching experience at the university and industrial experience in Japan. He has several publications in the top-ranked international journals and international conferences. He was a Lecturer in the Department of Computer Science and Engineering at the University of Asia Pacific, Bangladesh. He also served Fujitsu Research Institute, Japan, as a Data Analyst Engineer; University of Information Technology and Sciences, Bangladesh, as a Lecturer; and the ICT division, Bangladesh, as a Research Fellow. Tanzir has achieved an academic genius award (1st position) both in B.Sc. (Engg.) and M.Sc. (Engg.) degrees from Mawlana Bhashani Science and Technology University, Bangladesh, for his academic excellence. His research interests include cybersecurity and machine learning.



Dr. Chadni Islam is a Lecturer of Cybersecurity at Edith Cowan University (ECU), a leading institution in cybersecurity education and research in Australia. Before joining ECU, Dr. Islam was a Lecturer of Cybersecurity at the Queensland University of Technology (QUT) and a Postdoctoral Research Fellow at the University of Adelaide. She completed her PhD at the University of Adelaide in collaboration with CSIRO's Data61, focusing on the intersection of cybersecurity and software engineering. Her research explores the integration of machine learning, natural language processing, and cybersecurity to develop innovative security solutions. Passionate about both teaching and research, she is dedicated to advancing cybersecurity knowledge and fostering technological resilience in digital systems.



Dr. Gowri Ramachandran is a Senior Lecturer at the School of Information Systems. He began his career at QUT as a research fellow in the Trusted Networks Lab, School of Computer Science, Queensland University of Technology (QUT), under the mentorship of Prof. Raja Jurdak. Prior to joining QUT, he was a postdoctoral researcher at Viterbi School of Engineering, University of Southern California (USC), where he was associated with Autonomous Networks Research Group (ANRG) and Center for Cyber-Physical System and the Internet-of-Things (CCI) under the mentorship of Prof. Bhaskar Krishnamachari. Gowri's research interests are around cybersecurity, service computing, IoT, and distributed systems, with a focus on problems related to performance, automation, trust, and scalability.



Raja Jurdak (Senior Member, IEEE) received the M.S. and Ph.D. degrees from the University of California at Irvine. He is a Professor of distributed systems and the Chair of applied data sciences with Queensland University of Technology, and the Director of the Trusted Networks Laboratory. He is also an Adjunct Professor with the University of New South Wales and a Visiting Researcher with Data61, CSIRO. Previously, he established and led the Distributed Sensing Systems Group, Data61, CSIRO. He also spent time as a Visiting Academic with MIT and Oxford University in 2011 and 2017, respectively. He has published over 300 peer reviewed publications, including three authored books on IoT, blockchain, and cyberphysical systems. His publications have attracted over 17,200 citations, with an H-index of 56. His research interests include trust, mobility, and energy efficiency in networks. He serves on the organizing and technical program committees of top international conferences, including Percom, ICBC, IPSN, WoWMoM, and ICDCS. He serves on the editorial boards of Ad Hoc Networks and Scientific Reports (Nature). He is an IEEE Computer Society Distinguished Visitor.

Supplementary Materials of "DySec: A Machine Learning-based Dynamic Analysis for Detecting Malicious Packages in PyPI Ecosystem"

Sk Tanzir Mehedi, Chadni Islam, Gowri Ramachandran, and Raja Jurdak

Appendix A1: Detailed description of the dataset with examples.

TC	Feature Name	Description	Examples/Instructions
*	Package_Name	Package Name and Version	1337z-4.4.7, 1337x-1.2.6
FiletopTraces	Read_Processes	Processes involved in reading	pip reads setup.py for metadata, MANIFEST.in for package data
	Write_Processes	Processes involved in writing	writes to site-packages and cached .whl to .cache/pip
	Read_Data_Transfer	Processes for reading data transfer	pip reads .whl file from PyPI via HTTPS
	Write_Data_Transfer	Processes for writing data transfer	pip writes downloaded .whl into the local cache
	File_Access_Processes	Processes accessing files	python accesses requests_init_.py during installation
InstallTraces	Total_Dependencies	Total number of dependencies	17 (attrs-24.2.0; beautifulsoup4-0.1; cattrs-24.1.2; certifi-2024.8.30; charset-normalizer-3.4.0; cloudscraper-1.2.71; idna-3.10; platformdirs-4.3.6; pyparsing-3.2.0; requests-2.32.3;)
	Direct_Dependencies	List of direct dependencies	4 (beautifulsoup4-0.1; cloudscraper<2.0.0,>=1.2.71; requests-cache<2.0.0,>=1.2.1; requests<3.0.0,>=2.32.3;)
	Indirect_Dependencies	List of indirect dependencies	13 (attrs>=21.2; cattrs>=22.2; certifi>=2017.4.17; charset-normalizer<4,>=2; idna<4,>=2.5; platformdirs>=2.5;)
OpensnooplTraces	Root_DIR_Access	Root directory access in instal-time	2 (/root/.config/pip/pip.conf, /root/.ssh/authorized_keys)
	Temp_DIR_Access	Temp directory access in instal-time	15 (/tmp/pip-wheel-pzrcqrtt/1337z-4.4.7-py3-none-any.whl)
	Home_DIR_Access	Home directory access in instal-time	55 (/home/TruNETS-QUT/Analysis/Env/1337z-4.4.7..)
	User_DIR_Access	User directory access in instal-time	226 (/usr/lib/python3.12/lib-dynload, /etc/hosts)
	Sys_DIR_Access	System directory access in instal-time	12 (/sys/kernel/net/ipv4/ip_forward)
	Etc_DIR_Access	Etc directory access in instal-time	116 (/etc/host.conf, /etc/nftables.conf, /etc/ufw/ufw.conf)
	Other_DIR_Access	Operations involving other directories	17 (/proc/sys/net/ipv4/conf, /.ssh)
TCPTraces	State_Transition	TCP connection lifecycle in install-time	{'CLOSE -> ->': 15, 'SYN_SENT -> ->': 15, 'ESTABLISHED -> ->': 15, 'FIN_WAIT1 -> ->': 14, 'FIN_WAIT2 -> ->': 13}
	Local_IPs_Access	Access local IPs during install	2 (192.168.0.105, 192.168.0.1)
	Remote_IPs_Access	Access remote IPs during install	3 (151.101.0.223, 3.164.36.120, 152.199.39.144)
	Local_Port_Access	Access local ports during install	3 (52904, 53158, 34214)
	Remote_Port_Access	Access remote ports during install	3 (443, 23, 6667)
SysCallTraces	IO_Operations	Input/Output operations performed	ioctl, poll, readv, writev
	File_Operations	File-related system calls	open, openat, creat, read, write, close, lseek
	Network_Operations	Network-related operations	socket, connect, accept, listen, send
	Time_Operations	Time-based operations	clock_gettime, time, timer_create, timer_delete, alarm
	Security_Operations	Security-related system calls	getuid, setuid, geteuid, getgid, setgid, capset, capget, chmod
	Process_Operations	Process management system calls	fork, vfork, clone, wait4, waitpid, exit, kill, getpid, getppid
PatternTraces	Pattern_1	File metadata retrieval	newfstatat -> openat -> fstat
	Pattern_2	Reading data from a file	read -> pread64 -> lseek
	Pattern_3	Writing data to a file	write -> pwrite64 -> fsync
	Pattern_4	Network socket creation	socket -> bind -> listen
	Pattern_5	Creating a new process	fork -> execve -> wait4
	Pattern_6	Memory mapping	mmap -> mprotect -> munmap -> no-error -> no-fd
	Pattern_7	File descriptor management	dup -> dup2 -> close -> no-error -> stdout
	Pattern_8	Inter-process communication	pipe -> write -> read -> no-error -> pipe-fd
	Pattern_9	File locking	fcntl -> lockf -> close -> error=EACCES -> file-fd
	Pattern_10	Error handling	open -> read -> error=ENOENT -> no-error -> no-fd
*	Lables	Classification level	[1,0]